

# MINERÍA Y EL ATAQUE DEL 51 % PASO A PASO

La teoría de la Sesión 4, convertida en código Python que corre en tu computador.  
Sin red, sin billetera, sin internet. Solo Python y estas instrucciones.

| MODALIDAD                         | REQUISITO                        | DURACIÓN                          | META                           |
|-----------------------------------|----------------------------------|-----------------------------------|--------------------------------|
| <b>Individual<br/>o en pareja</b> | <b>Python 3.10+<br/>y pytest</b> | <b>2 a 3 horas<br/>a tu ritmo</b> | <b>32 pruebas<br/>en verde</b> |

## CÓMO LEER ESTA GUÍA

Está pensada para que puedas hacer el laboratorio **sin ayuda de nadie**, incluso si es la primera vez que abres una terminal. Cada paso dice exactamente qué escribir y qué deberías ver. Si algo no sale como aquí se describe, ve al final: **«Errores frecuentes»** resuelve casi todo. No hay que memorizar nada: se trata de entender, escribir unas pocas líneas y comprobar con las pruebas.

## 01 Qué vas a construir, y por qué

[LEER PRIMERO](#)

En la Sesión 4 vimos, en diapositivas, cómo una red de desconocidos se pone de acuerdo en un solo orden de los hechos. Hoy eso deja de ser una explicación y pasa a ser código que tú ejecutas. Vas a programar **tres piezas**, cada una atada a una lámina concreta de la clase:

| PARTE                  | QUÉ PROGRAMAS                                                                          | QUÉ LÁMINA FIJA |
|------------------------|----------------------------------------------------------------------------------------|-----------------|
| 1 · Prueba de trabajo  | Buscar un <code>nonce</code> cuyo hash quede por debajo de un objetivo, y verificarlo. | A.5 y A.5b      |
| 2 · Elección de cadena | Decidir cuál de dos cadenas gana, por trabajo acumulado.                               | A.7             |
| 3 · El ataque del 51 % | La fórmula del whitepaper de Bitcoin, una simulación, y el cálculo exacto.             | A.8, A.8b y A.9 |

#### EL HALLAZGO QUE HACE ESTO INTERESANTE

La tabla famosa del whitepaper de Bitcoin (la de «6 confirmaciones») es una **aproximación**, y subestima el riesgo real. En la Parte 3 vas a programar la fórmula original, después una simulación honesta, y vas a **descubrir que no coinciden**: hace falta esperar más confirmaciones de las que dice el whitepaper. Es la misma lección de la actividad de Estonia: un documento célebre, repetido mil veces, puede tener un error — y tú lo compruebas con código en lugar de creerlo.

No necesitas entender toda la matemática de antemano. Cada función que tienes que completar trae, en su comentario, la fórmula o la idea exacta. Tu trabajo es traducirla a Python y dejar que las pruebas confirmen que quedó bien.

## 02 Instalar Python (si no lo tienes)

UNA SOLA VEZ

Necesitas **Python 3.10 o más nuevo**. Primero comprueba si ya lo tienes. Abre una terminal:

- **Windows:** tecla Windows, escribe `cmd`, Enter. (O «PowerShell».)
- **macOS:** Cmd + Espacio, escribe `Terminal`, Enter.
- **Linux:** Ctrl + Alt + T.

Y escribe (en Windows prueba primero `python`; en macOS y Linux, `python3`):

```
python --version
```

Si responde algo como `Python 3.12.1`, ya lo tienes: salta al paso 3. Si dice «no se reconoce» o muestra una versión menor que 3.10, instálalo:

| SISTEMA | CÓMO INSTALAR                                                                                                                                                                                                 |
|---------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Windows | Ve a <code>python.org/downloads</code> , descarga el instalador y — <b>MUY IMPORTANTE</b> — marca la casilla «Add Python to PATH» antes de pulsar Install. Sin esa casilla, la terminal no encontrará Python. |
| macOS   | Descárgalo de <code>python.org/downloads</code> , o con Homebrew: <code>brew install python</code> .                                                                                                          |
| Linux   | Casi siempre ya viene. Si no: <code>sudo apt install python3 python3-pip</code> (Debian/Ubuntu).                                                                                                              |

Cierra y vuelve a abrir la terminal después de instalar, y comprueba otra vez con `python --version`.

### INSTALAR pytest

El laboratorio usa `pytest` para las pruebas. Instálalo una vez:

```
python -m pip install pytest
```

Si `python` no funciona pero `python3` sí, usa `python3 -m pip install pytest`. Úsalo así (con `python3`) en todos los comandos siguientes.

## 03 La carpeta del laboratorio

Todo está en `material/laboratorio-04`. Su contenido:

```
laboratorio-04/
├── consenso.py      <- AQUÍ TRABAJAS: tiene funciones con TODO
├── test_consenso.py <- las 32 pruebas. NO se modifica
├── andamiaje/       <- copia intacta de consenso.py, por si empiezas de cero
└── solucion/        <- la solución de referencia (míralo solo al final)
```

Abre una terminal **dentro** de esa carpeta. La forma más simple:

- **Windows:** abre la carpeta en el Explorador, haz clic en la barra de dirección, escribe `cmd` y Enter.
- **macOS/Linux:** `cd` y arrastra la carpeta a la terminal, luego Enter. Ejemplo: `cd "ruta/a/material/laboratorio-04"`.

Comprueba que estás en el sitio correcto listando los archivos: `dir` en Windows, `ls` en macOS/Linux. Debes ver `consenso.py` y `test_consenso.py`.

#### EL FLUJO: TE GUÍAN LAS PRUEBAS

No tienes que adivinar si tu código está bien. Corre las pruebas y ellas te dicen. Al empezar, **las 32 fallan** (rojo). Vas completando funciones y volviendo a correr; el laboratorio está terminado cuando las 32 pasan (verde). Corre las pruebas así:

```
python -m pytest -q
```

Para trabajar por partes y no ver el mar de errores completo, filtra por nombre:

```
python -m pytest -q -k "objetivo or minar or nonce or bits" # Parte 1
python -m pytest -q -k "gana or empate" # Parte 2
python -m pytest -q -k "formula or confirmaciones or mayoria" # Parte 3a
python -m pytest -q -k "simulacion or subestima" # Parte 3b
```

Recuerda el ejemplo real de la lámina A.5b: un bloque «gana» si el hash de su encabezado, con cierto `nonce`, queda por debajo de un objetivo. Minar es probar `nonce` tras `nonce` hasta encontrarlo. Verificar es recalcular un solo hash.

1

FUNCIÓN · `objetivo_para(bits)`

El umbral que el hash debe no superar

**Qué hace:** con `bits` de dificultad, el objetivo es  $2^{(256 - \text{bits})}$ . Un hash gana si, leído como número, es menor que ese objetivo.

**Qué escribir:** una sola línea que devuelva `2 ** (256 - bits)`.

**Pista:** en Python, `2 ** n` es «2 elevado a n». No hace falta nada más.

**Cómo lo comprobas:** con 0 bits, el objetivo es  $2^{256}$  (cualquier hash gana). Cada bit extra parte el objetivo a la mitad.

2

FUNCIÓN · `minar(encabezado, bits)`

Buscar el nonce ganador

**Qué hace:** prueba `nonce = 0, 1, 2, ...` y devuelve el **primero** cuyo `sha256(encabezado + nonce)` quede por debajo del objetivo.

**Qué escribir:** un bucle. En cada vuelta: calcula el hash del texto `encabezado + str(nonce)`, conviértelo a número con `int(hash_hex, 16)`, y si es menor que `objetivo_para(bits)`, devuelve un `ResultadoMinado`.

**Pistas:**

- El hash en hexadecimal: `hashlib.sha256((encabezado + str(nonce)).encode()).hexdigest()`
- `int(hash_hex, 16)` lo vuelve un entero comparable con el objetivo.
- `ResultadoMinado` ya está definido arriba: guarda `nonce`, `hash_hex`, `intentos` (que es `nonce + 1`) y `segundos` (mide el tiempo con `time.perf_counter()` al inicio y al final).

**Cómo lo comprobas:** el resultado que devuelvas debe pasar `verificar_minado` (la función siguiente).

3

FUNCIÓN · `verificar_minado(encabezado, bits, nonce)`

Comprobar en un solo hash

**Qué hace:** recalcula `sha256(encabezado + nonce)` UNA vez y devuelve `True` si queda por debajo del objetivo.

**Qué escribir:** el mismo cálculo del hash que en `minar`, pero sin bucle: un solo hash, comparar, devolver `True` o `False`.

**La idea que demuestra:** producir el nonce costó muchos intentos; verificarlo cuesta uno. Esa asimetría es toda la prueba de trabajo.

Cuando las tres estén listas, corre `python -m pytest -q -k "objetivo or minar or nonce or bits"`. Deben pasar en verde. Verás pruebas como «una transferencia simple», «más bits exige más intentos» y «el nonce no sirve para otro encabezado».

Recuerda la lámina A.7: cuando hay dos cadenas válidas, gana la de **más trabajo acumulado**, no la más larga. El trabajo de un bloque es  $2^{\text{bits}}$  (ya está calculado en la propiedad `trabajo` de la clase `Bloque`).

4

FUNCIÓN · `trabajo_acumulado(cadena)`

Sumar el trabajo de todos los bloques

**Qué hace:** devuelve la suma del `trabajo` de cada bloque de la cadena.

**Qué escribir:** una línea. **Pista:** `sum(b.trabajo for b in cadena)`.

5

FUNCIÓN · `elegir_cadena(a, b)`

Quedarse con la de más trabajo

**Qué hace:** devuelve la cadena de mayor trabajo acumulado. Ante un **empate**, conserva la primera (`a`): es lo que hace Bitcoin, no cambiar de rama sin una razón.

**Qué escribir:** compara los dos trabajos acumulados. Devuelve `b` solo si su trabajo es **estrictamente mayor** que el de `a`; si no, devuelve `a`.

**Pista:** `return b if trabajo_acumulado(b) > trabajo_acumulado(a) else a`.

**El detalle que evalúan las pruebas:** una cadena más CORTA pero con bloques más difíciles puede ganar. Y el empate se resuelve a favor de `a`.

Corre `python -m pytest -q -k "gana or empate"`. Verás «gana el mayor trabajo aunque sea más corta», «con igual dificultad gana la más larga» y «en empate se conserva la actual».

PREGUNTA PARA PENSAR (VA EN EL INFORME SI TU DOCENTE LO PIDE)

¿Por qué un nodo no cambia de cadena ante un empate? Piensa qué pasaría en la red si todos cambiaran de rama cada vez que ven una igual de pesada.

Aquí está el corazón del laboratorio y el hallazgo. Vas a calcular la probabilidad de que un atacante con una fracción  $q$  del cómputo alcance a la cadena honesta cuando el comerciante esperó  $z$  confirmaciones — de **tres** formas distintas, y a compararlas.

6

FUNCIÓN · `probabilidad_nakamoto(q, z)`

La fórmula original del whitepaper

**Qué hace:** traduce a Python el programa en C de la sección 11 del whitepaper de Bitcoin. El comentario de la función trae la fórmula exacta.

**Cómo se arma:**  $p = 1 - q$ ;  $\text{lam} = z * (q / p)$ ; luego un sumatorio para  $k$  de 0 a  $z$  de `Poisson(k; lam) · (1 - (q/p)^(z-k))`, y la probabilidad es  $1 -$  ese sumatorio.

**Pistas:**

- El término de Poisson: empieza en `math.exp(-lam)` y multiplícalo por `lam / i` dentro de un bucle `i` de 1 a  $k$  (así evitas factoriales grandes).
- Casos borde: si  $q \geq p$  (atacante con la mitad o más), devuelve `1.0`.

**Cómo lo comprobamos:** las pruebas comparan tu resultado con **los números que Nakamoto publicó**. Si tu traducción es correcta, coinciden hasta el octavo decimal. Por ejemplo, con  $q = 0,10$  y  $z = 1$  debe dar `0,2045873`.

7

FUNCIÓN · `simular_ataque(q, z)`

La misma probabilidad, sin fórmula

**Qué hace:** en vez de una fórmula, **simula** la carrera miles de veces y cuenta cuántas gana el atacante. El comentario describe las dos fases.

**Cómo se arma:** repite `ensayos` veces. En cada ensayo: mina bloques hasta que los honestos llevan  $z$  (el atacante lleva algunos en secreto); luego el «déficit» =  $z - (\text{bloques del atacante})$  hace un paseo aleatorio: baja 1 con probabilidad  $q$ , sube 1 con probabilidad  $p$ . Si el déficit llega a 0, el atacante ganó ese ensayo. Devuelve la fracción de ensayos ganados.

**Pistas:**

- Usa `rnd = random.Random(semilla)` y `rnd.random() < q` para decidir cada bloque.
- Si el déficit supera el `tope`, da el ensayo por perdido (volver es despreciable).

**Lo que vas a notar:** la simulación NO coincide con la fórmula del whitepaper. Sale sistemáticamente **más alta**. Eso no es un error tuyo: es el hallazgo.

Por qué el *whitepaper* se queda corto

**Qué hace:** el cálculo exacto (Rosenfeld, 2014), sin la aproximación de Poisson que usó Nakamoto. El comentario trae la fórmula:  $P = 1 - \sum C(m+z-1, m) \cdot (p^z q^m - q^z p^m)$  para  $m$  de 0 a  $z$ .

**Pistas:**

- $C(m+z-1, m)$  es una combinatoria: `math.comb(m + z - 1, m)`.
- Con  $q$  grande y  $z$  grande, esa combinatoria desborda un `float`. Para evitarlo, calcula en espacio logarítmico con `math.lgamma` (la guía de la solución muestra cómo; si no llegas ahí, el cálculo directo con `math.comb` pasa las pruebas para los valores que se prueban).

**La confirmación:** la simulación de la función 7 debe coincidir con esta fórmula exacta, no con la de Nakamoto.

## EL RESULTADO QUE CIERRA EL LABORATORIO

La función `confirmaciones_necesarias` (que ya viene hecha) calcula cuántas confirmaciones hacen falta para que el riesgo baje de 0,1 %. Al comparar la fórmula del *whitepaper* con la exacta, el *whitepaper* pide menos confirmaciones de las necesarias:

| ATACANTE (q) | WHITEPAPER | EXACTA (ROSENFELD) |
|--------------|------------|--------------------|
| 10 %         | 5          | 6                  |
| 20 %         | 11         | 13                 |
| 30 %         | 24         | 32                 |
| 40 %         | 89         | 133                |

Con un atacante del 30 %, esperar las 24 confirmaciones que sugiere el *whitepaper* **no basta**: hacen falta 32. Lo demostraste tú, con código.

## 07 La prueba final: todo junto

Cuando las 32 pruebas pasen ( `python -m pytest -q muestra 32 passed` ), corre el experimento completo:

```
python consenso.py
```

Deberías ver algo muy parecido a esto (los números de la Parte 1 y de la simulación varían un poco por el azar; los de la fórmula y las confirmaciones son fijos):

```
PARTE 1 · dificultad frente a intentos
bits   intentos prom.   esperados 2^bits   segundos
8      92              256             0.0002
12     5,433           4,096             0.0160
16     88,918          65,536            0.1347
18    177,044          262,144            0.2639

PARTE 3 · whitepaper, formula exacta y simulacion (q = 0,30)
z= 0  nakamoto=1.0000  exacta=1.0000  simulacion=1.0000
z= 5  nakamoto=0.1774  exacta=0.1976  simulacion=0.1937
z=10  nakamoto=0.0417  exacta=0.0651  simulacion=0.0663
z=15  nakamoto=0.0101  exacta=0.0233  simulacion=0.0267

Confirmaciones para P < 0,1 %      whitepaper   exacta
q=0.10                             5             6
q=0.30                             24            32
q=0.45                             340           539
```

Fíjate en la Parte 3: la columna `nakamoto` siempre es menor que `exacta`, y `simulacion` se parece a `exacta`, no a `nakamoto`. Ese es, en tres columnas, todo el punto del laboratorio.

SI TE PIERDES O QUIERES EMPEZAR DE CERO

Copia otra vez el andamiaje sobre tu archivo de trabajo:

```
copy andamiaje\consenso.py consenso.py      (Windows)
cp andamiaje/consenso.py consenso.py        (macOS / Linux)
```

Y si quieres comparar con la solución de referencia, está en `solucion/consenso.py`. Míralo **solo al final**: el aprendizaje está en intentarlo tú primero.

## 08 Errores frecuentes

CASI TODO SE RESUELVE AQUÍ

| SÍNTOMA                                                                                | CAUSA Y SOLUCIÓN                                                                                                                                   |
|----------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| «python no se reconoce como comando»                                                   | Python no está en el PATH. En Windows, reinstala marcando «Add Python to PATH». O prueba con <code>py</code> o <code>python3</code> .              |
| «No module named pytest»                                                               | Falta instalarlo: <code>python -m pip install pytest</code> .                                                                                      |
| «No such file or directory: consenso.py»                                               | La terminal no está en la carpeta del laboratorio. Usa <code>cd</code> para entrar en <code>material/laboratorio-04</code> .                       |
| <code>minar</code> nunca termina / se queda pensando                                   | Estás comparando el hash como texto, no como número. Convierte con <code>int(hash_hex, 16)</code> antes de comparar con el objetivo.               |
| La prueba de la tabla de Nakamoto falla por muy poco                                   | El sumatorio debe incluir <code>k = z</code> : usa <code>range(z + 1)</code> , no <code>range(z)</code> .                                          |
| La simulación da siempre 1.0                                                           | El paseo aleatorio no tiene salida por el <code>tope</code> , o invertiste el signo del paso (baja con <code>q</code> , sube con <code>p</code> ). |
| <code>OverflowError</code> en <code>probabilidad_exacta</code> con <code>q</code> alto | <code>math.comb</code> crece demasiado. Calcula en espacio logarítmico con <code>math.lgamma</code> (ver la solución de referencia).               |
| La simulación coincide con Nakamoto, no con la exacta                                  | Estás generando los bloques del atacante con una Poisson en vez de bloque a bloque. Simula el paseo, no la fórmula.                                |

## 09 Si quieres ir más allá

OPCIONAL

La función `experimento_dificultad` ya mide cuántos intentos toma minar según la dificultad. Con esos datos puedes hacer una gráfica del crecimiento exponencial (la idea de la lámina A.5b: cada bit extra duplica el trabajo). Si la traes a la Sesión 5, hay un punto extra de participación.

### POR QUÉ ESTE LABORATORIO IMPORTA, AUNQUE NO SE CALIFIQUE

Cada función que completaste es una diapositiva de la clase vuelta código: minar es A.5, elegir la cadena es A.7, el ataque es A.8 y A.9. El Quiz 1 pregunta justo esto, así que quien hizo el laboratorio responde sin dudar. Y el hallazgo de la Parte 3 es el método que atraviesa todo el curso: **verificar en lugar de creer**.

## 10 Referencias

- Nakamoto, S. (2008). *Bitcoin: A Peer-to-Peer Electronic Cash System*, sección 11.  
<https://bitcoin.org/bitcoin.pdf>
- Rosenfeld, M. (2014). *Analysis of hashrate-based double spending*. `arXiv:1402.2009`
- Grunspan, C. y Pérez-Marco, R. (2017). *Double spend races*. `arXiv:1702.02867`